

Chapter 14

Matrix Inversion

*It is amusing to remark that we were so involved with
matrix inversion that we probably talked of nothing else for months.
Just in this period Mrs. von Neumann acquired a big,
rather wild but gentle Irish Setter puppy,
which she called Inverse in honor of our work!*

— HERMAN H. GOLDSTINE, *The Computer: From Pascal to von Neumann* (1972)

*The most computationally intensive portion of the tasks assigned to the processors is
integrating the KKR matrix inverse over the first Brillouin zone.*

*To evaluate the integral,
hundreds or possibly thousands of complex double precision matrices
of order between 80 and 300 must be formed and inverted.
Each matrix corresponds to a different vertex of the tetrahedrons
into which the Brillouin zone has been subdivided.*

— M. T. HEATH, G. A. GEIST, and J. B. DRAKE,
*Superconductivity Computations,
in Early Experience with the Intel iPSC/860
at Oak Ridge National Laboratory* (1990)

*Press $\left(\frac{1}{x}\right)$ to invert the matrix.
Note that matrix inversion can produce erroneous results
if you are using ill-conditioned matrices.*

— HEWLETT-PACKARD, *HP 48G Series User's Guide* (1993)

Almost anything you can do with A^{-1} can be done without it.

— GEORGE E. FORSYTHE and CLEVE B. MOLER,
Computer Solution of Linear Algebraic Systems (1967)

14.1. Use and Abuse of the Matrix Inverse

To most numerical analysts, matrix inversion is a sin. Forsythe, Malcolm, and Moler put it well when they say [430, 1977, p. 31] “In the vast majority of practical computational problems, it is unnecessary and inadvisable to actually compute A^{-1} .” The best example of a problem in which the matrix inverse should not be computed is the linear equations problem $Ax = b$. Computing the solution as $x = A^{-1} \times b$ requires $2n^3$ flops, assuming A^{-1} is computed by Gaussian elimination with partial pivoting (GEPP), whereas GEPP applied directly to the system costs only $2n^3/3$ flops.

Not only is the inversion approach three times more expensive, but it is much less stable. Suppose $X = A^{-1}$ is formed *exactly*, and that the only rounding errors are in forming $x = fl(Xb)$. Then $\hat{x} = (X + \Delta X)b$, where $|\Delta X| \leq \gamma_n |X|$, by (3.11). So $A\hat{x} = A(X + \Delta X)b = (I + A\Delta X)b$, and the best possible residual bound is

$$|b - A\hat{x}| \leq \gamma_n |A| |A^{-1}| |b|.$$

For GEPP, Theorem 9.4 yields

$$|b - A\hat{x}| \leq \gamma_{3n} |\hat{L}| |\hat{U}| |\hat{x}|.$$

Since it is usually true that $\|\hat{L}\|\|\hat{U}\|_\infty \approx \|A\|_\infty$ for GEPP, we see that the matrix inversion approach is likely to give a much larger residual than GEPP if A is ill conditioned and if $\|x\|_\infty \ll \|A^{-1}\| \|b\|_\infty$. For example, we solved fifty 25×25 systems $Ax = b$ in MATLAB, where the elements of x are taken from the normal $N(0, 1)$ distribution and A is random with $\kappa_2(A) = u^{-1/2} \approx 9 \times 10^7$. As Table 14.1 shows, the inversion approach provided much larger backward errors than GEPP in this experiment.

Given the inexpedience of matrix inversion, why devote a chapter to it? The answer is twofold. First, there *are* situations in which a matrix inverse must be computed. Examples are in statistics [63, 1974, §7.5], [806, 1984, §2.3], [834, 1989, p. 342 ff], where the inverse can convey important statistical information, in certain matrix iterations arising in eigenvalue-related problems [47, 1997], [193, 1987], [621, 1994], in the computation of matrix functions such as the square root [609, 1997] and the logarithm [231, 2001], and in numerical integrations arising in superconductivity computations [555, 1990] (see the quotation at the start of the chapter). Second, methods for matrix inversion display a wide variety of stability properties, making for instructive and challenging error analysis. (Indeed, the first major rounding error analysis to be published, that of von Neumann and Goldstine, was for matrix inversion—see §9.13).

Table 14.1. Backward errors $\eta_{A,b}(\hat{x})$ for the ∞ -norm.

	min	max
$x = A^{-1} \times b$	6.66e-12	1.69e-10
GEPP	3.44e-18	7.56e-17

Matrix inversion can be done in many different ways—in fact, there are more computationally distinct possibilities than for any other basic matrix computation. For example, in triangular matrix inversion different loop orderings are possible and either triangular matrix–vector multiplication, solution of a triangular system, or a rank-1 update of a rectangular matrix can be employed inside the outer loop. More generally, given a factorization $PA = LU$, two ways to evaluate A^{-1} are as $A^{-1} = U^{-1} \times L^{-1} \times P$, and as the solution to $UA^{-1} = L^{-1} \times P$. These methods generally achieve different levels of efficiency on high-performance computers, and they propagate rounding errors in different ways. We concentrate in this chapter on the numerical stability, but comment briefly on performance issues.

The quality of an approximation $Y \approx A^{-1}$ can be assessed by looking at the right and left residuals, $AY - I$ and $YA - I$, and the forward error, $Y - A^{-1}$. Suppose we perturb $A \rightarrow A + \Delta A$ with $|\Delta A| \leq \epsilon|A|$; thus, we are making relative perturbations of size at most ϵ to the elements of A . If $Y = (A + \Delta A)^{-1}$ then $(A + \Delta A)Y = Y(A + \Delta A) = I$, so that

$$|AY - I| = |\Delta AY| \leq \epsilon|A||Y|, \quad (14.1)$$

$$|YA - I| = |Y\Delta A| \leq \epsilon|Y||A|, \quad (14.2)$$

and, since $(A + \Delta A)^{-1} = A^{-1} - A^{-1}\Delta AA^{-1} + O(\epsilon^2)$,

$$|A^{-1} - Y| \leq \epsilon|A^{-1}||A||A^{-1}| + O(\epsilon^2). \quad (14.3)$$

(Note that (14.3) can also be derived from (14.1) or (14.2).) The bounds (14.1)–(14.3) represent “ideal” bounds for a computed approximation Y to A^{-1} , if we regard ϵ as a small multiple of the unit roundoff u . We will show that, for triangular matrix inversion, appropriate methods do indeed achieve (14.1) or (14.2) (but not both) and (14.3).

It is important to note that neither (14.1), (14.2), nor (14.3) implies that $Y + \Delta Y = (A + \Delta A)^{-1}$ with $\|\Delta A\|_\infty \leq \epsilon\|A\|_\infty$ and $\|\Delta Y\|_\infty \leq \epsilon\|Y\|_\infty$, that is, Y need not be close to the inverse of a matrix near to A , even in the norm sense. Indeed, such a result would imply that both the left and right residuals are bounded in norm by $(2\epsilon + \epsilon^2)\|A\|_\infty\|Y\|_\infty$, and this is not the case for any of the methods we will consider.

To illustrate the latter point we give a numerical example. Define the matrix $A_n \in \mathbb{R}^{n \times n}$ as `triu(qr(vand(n)))`, in MATLAB notation (`vand` is a routine from the Matrix Computation Toolbox—see Appendix D); in other words, A_n is the upper triangular QR factor of the $n \times n$ Vandermonde matrix based on equispaced points on $[0, 1]$. We inverted A_n , for $n = 5:80$, using MATLAB’s `inv` function, which uses GEPP. The left and right normwise relative residuals

$$\text{res}_L = \frac{\|\hat{X}A - I\|_\infty}{\|\hat{X}\|_\infty\|A\|_\infty}, \quad \text{res}_R = \frac{\|A\hat{X} - I\|_\infty}{\|A\|_\infty\|\hat{X}\|_\infty},$$

are plotted in Figure 14.1. We see that while the left residual is always less than the unit roundoff, the right residual becomes large as n increases. These matrices are very ill conditioned (singular to working precision for $n \geq 20$), yet it is still reasonable to expect a small residual, and we will prove in §14.3.2 that the left residual must be small, independent of the condition number.

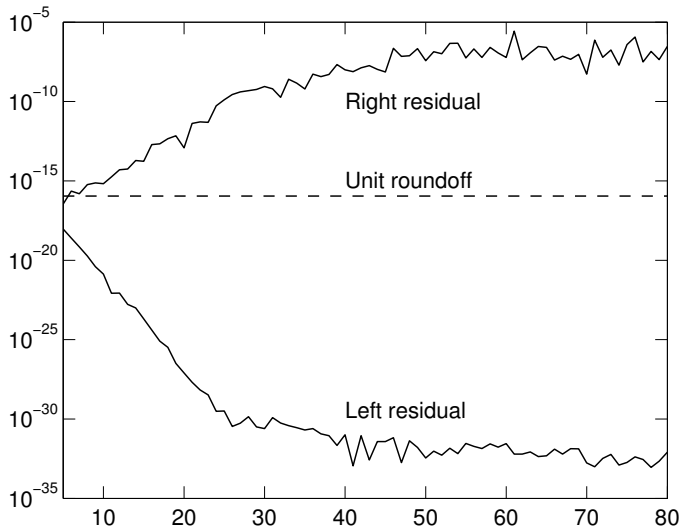


Figure 14.1. *Residuals for inverses computed by MATLAB's `inv` function.*

In most of this chapter we are not concerned with the precise values of constants (§14.4 is the exception); thus c_n denotes a constant of order n . To simplify the presentation we introduce a special notation. Let $A_i \in \mathbb{R}^{m_i \times n_i}$, $i = 1:k$, be matrices such that the product $A_1 A_2 \dots A_k$ is defined and let

$$p = \sum_{i=1}^{k-1} n_i.$$

Then $\Delta(A_1, A_2, \dots, A_k) \in \mathbb{R}^{m_1 \times n_k}$ denotes a matrix bounded according to

$$|\Delta(A_1, A_2, \dots, A_k)| \leq c_p u |A_1| |A_2| \dots |A_k|.$$

This notation is chosen so that if $\widehat{C} = fl(A_1 A_2 \dots A_k)$, with the product evaluated in any order, then

$$\widehat{C} = A_1 A_2 \dots A_k + \Delta(A_1, A_2, \dots, A_k).$$

14.2. Inverting a Triangular Matrix

We consider the inversion of a lower triangular matrix $L \in \mathbb{R}^{n \times n}$, treating unblocked and blocked methods separately. We do not make a distinction between partitioned and block methods in this section. All the results in this and the next section are from Du Croz and Higham [357, 1992].

14.2.1. Unblocked Methods

We focus our attention on two “ j ” methods that compute L^{-1} a column at a time. Analogous “ i ” and “ k ” methods exist, which compute L^{-1} row-wise or use outer products, respectively, and we comment on them at the end of the section.

The first method computes each column of $X = L^{-1}$ independently, using forward substitution. We write it as follows, to facilitate comparison with the second method.

Method 1.

```

for  $j = 1:n$ 
   $x_{jj} = l_{jj}^{-1}$ 
   $\bar{X}(j+1:n, j) = -x_{jj}L(j+1:n, j)$ 
  Solve  $L(j+1:n, j+1:n)\bar{X}(j+1:n, j) = \bar{X}(j+1:n, j)$ ,
  by forward substitution.
end

```

In BLAS terminology, this method is dominated by n calls to the level-2 BLAS routine **xTRSV** (TRiangular SolVe).

The second method computes the columns in the reverse order. On the j th step it multiplies by the previously computed inverse $L(j+1:n, j+1:n)^{-1}$ instead of solving a system with coefficient matrix $L(j+1:n, j+1:n)$.

Method 2.

```

for  $j = n:-1:1$ 
   $x_{jj} = l_{jj}^{-1}$ 
   $\bar{X}(j+1:n, j) = \bar{X}(j+1:n, j+1:n)L(j+1:n, j)$ 
   $\bar{X}(j+1:n, j) = -x_{jj}\bar{X}(j+1:n, j)$ 
end

```

Method 2 uses n calls to the level-2 BLAS routine **xTRMV** (TRiangular Matrix times Vector). On most high-performance machines **xTRMV** can be implemented to run faster than **xTRSV**, so Method 2 is generally preferable to Method 1 from the point of view of efficiency (see the performance figures at the end of §14.2.2). We now compare the stability of the two methods.

Theorem 8.5 shows that the j th column of the computed $\hat{X}$ from Method 1 satisfies

$$(L + \Delta L_j)\hat{x}_j = e_j, \quad |\Delta L_j| \leq c_n u |L|.$$

It follows that we have the componentwise residual bound

$$|L\hat{X} - I| \leq c_n u |L| |\hat{X}| \quad (14.4)$$

and the componentwise forward error bound

$$|\hat{X} - L^{-1}| \leq c_n u |L^{-1}| |L| |\hat{X}|. \quad (14.5)$$

Since $\hat{X} = L^{-1} + O(u)$, (14.5) can be written as

$$|\hat{X} - L^{-1}| \leq c_n u |L^{-1}| |L| |L^{-1}| + O(u^2), \quad (14.6)$$

which is invariant under row and column scaling of L . If we take norms we obtain normwise relative error bounds that are either row or column scaling independent: from (14.6) we have

$$\frac{\|\hat{X} - L^{-1}\|_\infty}{\|L^{-1}\|_\infty} \leq c_n u \operatorname{cond}(L^{-1}) + O(u^2), \quad (14.7)$$

and the same bound holds with $\text{cond}(L^{-1})$ replaced by $\text{cond}(L)$.

Notice that (14.4) is a bound for the *right residual*, $L\hat{X} - I$. This is because Method 1 is derived by solving $LX = I$. Conversely, Method 2 can be derived by solving $XL = I$, which suggests that we should look for a bound on the *left residual* for this method.

Lemma 14.1. *The computed inverse $\hat{X}$ from Method 2 satisfies*

$$|\hat{X}L - I| \leq c_n u |\hat{X}| |L|. \quad (14.8)$$

Proof. The proof is by induction on n , the case $n = 1$ being trivial. Assume the result is true for $n - 1$ and write

$$L = \begin{bmatrix} \alpha & 0 \\ y & M \end{bmatrix}, \quad X = L^{-1} = \begin{bmatrix} \beta & 0 \\ z & N \end{bmatrix},$$

where $\alpha, \beta \in \mathbb{R}$, $y, z \in \mathbb{R}^{n-1}$, and $M, N \in \mathbb{R}^{(n-1) \times (n-1)}$. Method 2 computes the first column of X by solving $XL = I$ according to

$$\beta = \alpha^{-1}, \quad z = -\beta Ny.$$

In floating point arithmetic we obtain

$$\begin{aligned} \hat{\beta} &= \alpha^{-1}(1 + \delta), & |\delta| &\leq u, \\ \hat{z} &= -\hat{\beta}\hat{N}y + \Delta(\hat{\beta}, \hat{N}, y). \end{aligned}$$

Thus

$$\begin{aligned} \hat{\beta}\alpha &= 1 + \delta, \\ \hat{z}\alpha + \hat{N}y &= -\delta\hat{N}y + \alpha\Delta(\hat{\beta}, \hat{N}, y). \end{aligned}$$

This may be written as

$$\begin{aligned} |\hat{X}L - I|(1:n, 1) &\leq \begin{bmatrix} u \\ u|\hat{N}||y| + c_n u(1 + u)|\hat{N}||y| \end{bmatrix} \\ &\leq c'_n u (|\hat{X}||L|)(1:n, 1). \end{aligned}$$

By assumption, the corresponding inequality holds for the $(2:n, 2:n)$ submatrices and so the result is proved. $\square$

Lemma 14.1 shows that Method 2 has a left residual analogue of the right residual bound (14.4) for Method 1. Since there is, in general, no reason to choose between a small right residual and a small left residual, our conclusion is that Methods 1 and 2 have equally good numerical stability properties.

More generally, it can be shown that all three i , j , and k inversion variants that can be derived from the equations $LX = I$ produce identical rounding errors under suitable implementations, and all satisfy the same right residual bound; likewise, the three variants corresponding to the equation $XL = I$ all satisfy the same left residual bound. The LINPACK routine `xTRDI` uses a k variant derived from $XL = I$; the LINPACK routines `xGEDI` and `xPODI` contain analogous code for inverting an upper triangular matrix (but the *LINPACK Users' Guide* [341, 1979, Chaps. 1 and 3] describes a different variant from the one used in the code).

14.2.2. Block Methods

Let the lower triangular matrix $L \in \mathbb{R}^{n \times n}$ be partitioned in block form as

$$L = \begin{bmatrix} L_{11} & & & \\ L_{21} & L_{22} & & \\ \vdots & & \ddots & \\ L_{N1} & \dots & \dots & L_{NN} \end{bmatrix}, \quad (14.9)$$

where we place no restrictions on the block sizes, other than to require the diagonal blocks to be square. The most natural block generalizations of Methods 1 and 2 are as follows. Here, we use the notation $L_{p:q,r:s}$ to denote the submatrix comprising the intersection of block rows p to q and block columns r to s of L .

Method 1B.

```

for  $j = 1:N$ 
   $X_{jj} = L_{jj}^{-1}$  (by Method 1)
   $X_{j+1:N,j} = -L_{j+1:N,j} X_{jj}$ 
  Solve  $L_{j+1:N,j+1:N} X_{j+1:N,j} = X_{j+1:N,j}$ ,
  by forward substitution
end
```

Method 2B.

```

for  $j = N:-1:1$ 
   $X_{jj} = L_{jj}^{-1}$  (by Method 2)
   $X_{j+1:N,j} = X_{j+1:N,j+1:N} L_{j+1:N,j}$ 
   $X_{j+1:N,j} = -X_{j+1:N,j} X_{jj}$ 
end
```

One can argue that Method 1B carries out the same arithmetic operations as Method 1, although possibly in a different order, and that it therefore satisfies the same error bound (14.4). For completeness, we give a direct proof.

Lemma 14.2. *The computed inverse $\hat{X}$ from Method 1B satisfies*

$$|L\hat{X} - I| \leq c_n u |L| |\hat{X}|. \quad (14.10)$$

Proof. Equating block columns in (14.10), we obtain the N independent inequalities

$$|L\hat{X}_{1:N,j} - I_{1:N,j}| \leq c_n u |L| |\hat{X}_{1:N,j}|, \quad j = 1:N. \quad (14.11)$$

It suffices to verify the inequality with $j = 1$. Write

$$L = \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix}, \quad X = \begin{bmatrix} X_{11} & 0 \\ X_{21} & X_{22} \end{bmatrix},$$

where $L_{11}, X_{11} \in \mathbb{R}^{r \times r}$ and L_{11} is the $(1,1)$ block in the partitioning of (14.9). X_{11} is computed by Method 1 and so, from (14.4),

$$|L_{11}\hat{X}_{11} - I| \leq c_r u |L_{11}| |\hat{X}_{11}| = c_r u (|L| |\hat{X}|)_{11}. \quad (14.12)$$

X_{21} is computed by forming $T = -L_{21}X_{11}$ and solving $L_{22}X_{21} = T$. The computed $\hat{X}_{21}$ satisfies

$$L_{22}\hat{X}_{21} + \Delta(L_{22}, \hat{X}_{21}) = -L_{21}\hat{X}_{11} + \Delta(L_{21}, \hat{X}_{11}).$$

Hence

$$\begin{aligned} |L_{21}\hat{X}_{11} + L_{22}\hat{X}_{21}| &\leq c_n u (|L_{21}||\hat{X}_{11}| + |L_{22}||\hat{X}_{21}|) \\ &= c_n u (|L||\hat{X}|)_{21}. \end{aligned} \quad (14.13)$$

Together, inequalities (14.12) and (14.13) are equivalent to (14.11) with $j = 1$, as required. $\square$

We can attempt a similar analysis for Method 2B. With the same notation as above, X_{21} is computed as $X_{21} = -X_{22}L_{21}X_{11}$. Thus

$$\hat{X}_{21} = -\hat{X}_{22}L_{21}\hat{X}_{11} + \Delta(\hat{X}_{22}, L_{21}, \hat{X}_{11}). \quad (14.14)$$

To bound the left residual we have to postmultiply by L_{11} and use the fact that X_{11} is computed by Method 2:

$$\hat{X}_{21}L_{11} + \hat{X}_{22}L_{21}(I + \Delta(\hat{X}_{11}, L_{11})) = \Delta(\hat{X}_{22}, L_{21}, \hat{X}_{11})L_{11}.$$

This leads to a bound of the form

$$|\hat{X}_{21}L_{11} + \hat{X}_{22}L_{21}| \leq c_n u |\hat{X}_{22}||L_{21}||\hat{X}_{11}||L_{11}|,$$

which would be of the desired form in (14.8) were it not for the factor $|\hat{X}_{11}||L_{11}|$. This analysis suggests that the left residual is not guaranteed to be small. Numerical experiments confirm that the left and right residuals can be large simultaneously for Method 2B, although examples are quite hard to find [357, 1992]; therefore the method must be regarded as unstable when the block size exceeds 1.

The reason for the instability is that there are *two* plausible block generalizations of Method 2 and we have chosen an unstable one that does not carry out the same arithmetic operations as Method 2. If we perform a solve with L_{jj} instead of multiplying by X_{jj} we obtain the second variation, which is used by LAPACK's xTRTRI.

Method 2C.

for $j = N: -1: 1$

$X_{jj} = L_{jj}^{-1}$ (by Method 2)

$X_{j+1:N,j} = X_{j+1:N,j+1:N}L_{j+1:N,j}$

Solve $X_{j+1:N,j}L_{jj} = -X_{j+1:N,j}$ by back substitution.

end

For this method, the analogue of (14.14) is

$$\hat{X}_{21}L_{11} + \Delta(\hat{X}_{21}, L_{11}) = -\hat{X}_{22}L_{21} + \Delta(\hat{X}_{22}, L_{21}),$$

which yields

$$|\hat{X}_{21}L_{11} + \hat{X}_{22}L_{21}| \leq c_n u (|\hat{X}_{21}||L_{11}| + |\hat{X}_{22}||L_{21}|).$$

Hence Method 2C enjoys a very satisfactory residual bound.

Table 14.2. *Mflop rates for inverting a triangular matrix on a Cray 2.*

		$n = 128$	$n = 256$	$n = 512$	$n = 1024$
Unblocked:	Method 1	95	162	231	283
	Method 2	114	211	289	330
	k variant	114	157	178	191
Blocked: (block size 64)	Method 1B	125	246	348	405
	Method 2C	129	269	378	428
	k variant	148	263	344	383

Lemma 14.3. *The computed inverse $\hat{X}$ from Method 2C satisfies*

$$|\hat{X}L - I| \leq c_n u |\hat{X}| |L|. \quad \square$$

In summary, block versions of Methods 1 and 2 are available that have the same residual bounds as the point methods. However, in general, there is no guarantee that stability properties remain unchanged when we convert a point method to block form, as shown by Method 2B.

In Table 14.2 we present some performance figures for inversion of a lower triangular matrix on a Cray 2. These clearly illustrate the possible gains in efficiency from using block methods, and also the advantage of Method 2 over Method 1. For comparison, the performance of a k variant is also shown (both k variants run at the same rate). The performance characteristics of the i variants are similar to those of the j variants, except that since they are row oriented rather than column oriented, they are liable to be slowed down by memory-bank conflicts, page thrashing, or cache missing.

14.3. Inverting a Full Matrix by LU Factorization

Next, we consider four methods for inverting a full matrix $A \in \mathbb{R}^{n \times n}$, given an LU factorization computed by GEPP. We assume, without loss of generality, that there are no row interchanges. We write the *computed* LU factors as L and U . Recall that $A + \Delta A = LU$, with $|\Delta A| \leq c_n u |L| |U|$ (Theorem 9.3).

14.3.1. Method A

Perhaps the most frequently described method for computing $X = A^{-1}$ is the following one.

Method A.
for $j = 1:n$
 Solve $Ax_j = e_j$.
end

Compared with the methods to be described below, Method A has the disadvantages of requiring more temporary storage and of not having a convenient

partitioned version. However, it is simple to analyse. From Theorem 9.4 we have

$$(A + \Delta A_j)\hat{x}_j = e_j, \quad |\Delta A_j| \leq c'_n u |L||U|, \quad (14.15)$$

and so

$$|A\hat{X} - I| \leq c'_n u |L||U||\hat{X}|. \quad (14.16)$$

This bound departs from the form (14.1) only in that $|A|$ is replaced by its upper bound $|L||U| + O(u)$. The forward error bound corresponding to (14.16) is

$$|\hat{X} - A^{-1}| \leq c'_n u |A^{-1}||L||U||\hat{X}|. \quad (14.17)$$

Note that (14.15) says that $\hat{x}_j$ is the j th column of the inverse of a matrix close to A , but it is a different perturbation ΔA_j for each column. It is not true that $\hat{X}$ itself is the inverse of a matrix close to A , unless A is well conditioned.

14.3.2. Method B

Next, we consider the method used by LINPACK's `xGEDI`, LAPACK's `xGETRI`, and MATLAB's `inv` function.

Method B.

Compute U^{-1} and then solve for X the equation $XL = U^{-1}$.

To analyse this method we will assume that U^{-1} is computed by an analogue of Method 2 or 2C for upper triangular matrices that obtains the columns of U^{-1} in the order 1 to n . Then the computed inverse $X_U \approx U^{-1}$ will satisfy the residual bound

$$|X_U U - I| \leq c_n u |X_U||U|.$$

We also assume that the triangular solve from the right with L is done by back substitution. The computed $\hat{X}$ therefore satisfies $\hat{X}L = X_U + \Delta(\hat{X}, L)$ and so

$$\hat{X}(A + \Delta A) = \hat{X}LU = X_U U + \Delta(\hat{X}, L)U.$$

This leads to the residual bound

$$\begin{aligned} |\hat{X}A - I| &\leq c_n u (|U^{-1}||U| + 2|\hat{X}||L||U|) \\ &\leq c'_n u |\hat{X}||L||U|, \end{aligned} \quad (14.18)$$

which is the left residual analogue of (14.16). From (14.18) we obtain the forward error bound

$$|\hat{X} - A^{-1}| \leq c'_n u |\hat{X}||L||U||A^{-1}|.$$

Note that Methods A and B are equivalent, in the sense that Method A solves for X the equation $LUX = I$ while Method B solves $XLU = I$. Thus the two methods carry out analogous operations but in different orders. It follows that the methods must satisfy analogous residual bounds, and so (14.18) can be deduced from (14.16).

We mention in passing that the LINPACK manual states that for Method B a bound holds of the form $\|A\hat{X} - I\| \leq d_n u \|A\| \|\hat{X}\|$ [341, 1979, p. 1.20]. This is incorrect, although counterexamples are rare; it is the *left* residual that is bounded this way, as follows from (14.18).

14.3.3. Method C

The next method that we consider is from Du Croz and Higham [357, 1992]. It solves the equation $UXL = I$, computing X a partial row and column at a time. To derive the method partition

$$X = \begin{bmatrix} x_{11} & x_{12}^T \\ x_{21} & X_{22} \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 \\ l_{21} & L_{22} \end{bmatrix}, \quad U = \begin{bmatrix} u_{11} & u_{12}^T \\ 0 & U_{22} \end{bmatrix},$$

where the $(1, 1)$ blocks are scalars, and assume that the trailing submatrix X_{22} is already known. Then the rest of X is computed according to

$$\begin{aligned} x_{21} &= -X_{22}l_{21}, \\ x_{12}^T &= -u_{12}^T X_{22}/u_{11}, \\ x_{11} &= 1/u_{11} - x_{12}^T l_{21}. \end{aligned}$$

The method can also be derived by forming the product $X = U^{-1} \times L^{-1}$ using the representation of L and U as a product of elementary matrices (and diagonal matrices in the case of U). In detail the method is as follows.

Method C.

for $k = n-1:1$

$$X(k+1:n, k) = -X(k+1:n, k+1:n)L(k+1:n, k)$$

$$X(k, k+1:n) = -U(k, k+1:n)X(k+1:n, k+1:n)/u_{kk}$$

$$x_{kk} = 1/u_{kk} - X(k, k+1:n)L(k+1:n, k)$$

end

The method can be implemented so that X overwrites L and U , with the aid of a work vector of length n (or a work array to hold a block row or column in the partitioned case). Because most of the work is performed by matrix–vector (or matrix–matrix) multiplication, Method C is likely to be the fastest of those considered in this section on many machines. (Some performance figures are given at the end of the section.)

A straightforward error analysis of Method C shows that the computed $\hat{X}$ satisfies

$$|U\hat{X}L - I| \leq c_n u |U| |\hat{X}| |L|. \quad (14.19)$$

We will refer to $U\hat{X}L - I$ as a “mixed residual”. From (14.19) we can obtain bounds on the left and right residual that are weaker than those in (14.18) and (14.16) by a factor $|U^{-1}||U|$ on the left or $|L||L^{-1}|$ on the right, respectively. We also obtain from (14.19) the forward error bound

$$|\hat{X} - A^{-1}| \leq c_n u |U^{-1}||U| |\hat{X}| |L| |L^{-1}|,$$

which is (14.17) with $|A^{-1}|$ replaced by its upper bound $|U^{-1}||L^{-1}| + O(u)$ and the factors reordered.

The LINPACK routine `xsIDI` uses a special case of Method C in conjunction with block LDL^T factorization with partial pivoting to invert a symmetric indefinite matrix; see Du Croz and Higham [357, 1992] for details.

14.3.4. Method D

The next method is based on another natural way to form A^{-1} and is used by LAPACK's xPOTRI, which inverts a symmetric positive definite matrix.

Method D.

Compute L^{-1} and U^{-1} and then form $A^{-1} = U^{-1} \times L^{-1}$.

The advantage of this method is that no extra workspace is needed; U^{-1} and L^{-1} can overwrite U and L , and can then be overwritten by their product. However, Method D is significantly slower on some machines than Methods B or C, because it uses a smaller average vector length for vector operations.

To analyse Method D we will assume initially that L^{-1} is computed by Method 2 (or Method 2C) and, as for Method B above, that U^{-1} is computed by an analogue of Method 2 or 2C for upper triangular matrices. We have

$$\hat{X} = X_U X_L + \Delta(X_U, X_L). \quad (14.20)$$

Since $A = LU - \Delta A$,

$$\hat{X}A = X_U X_L LU - X_U X_L \Delta A + \Delta(X_U, X_L)A. \quad (14.21)$$

Rewriting the first term of the right-hand side using $X_L L = I + \Delta(X_L, L)$, and similarly for U , we obtain

$$\hat{X}A - I = \Delta(X_U, U) + X_U \Delta(X_L, L)U - X_U X_L \Delta A + \Delta(X_U, X_L)A, \quad (14.22)$$

and so

$$\begin{aligned} |\hat{X}A - I| &\leq c'_n u (|U^{-1}||U| + 2|U^{-1}||L^{-1}||L||U| + |U^{-1}||L^{-1}||A|) \\ &\leq c''_n u |U^{-1}||L^{-1}||L||U|. \end{aligned} \quad (14.23)$$

This bound is weaker than (14.18), since $|\hat{X}| \leq |U^{-1}||L^{-1}| + O(u)$. Note, however, that the term $\Delta(X_U, X_L)A$ in the residual (14.22) is an unavoidable consequence of forming $X_U X_L$, and so the bound (14.23) is essentially the best possible.

The analysis above assumes that X_L and X_U both have small left residuals. If they both have small right residuals, as when they are computed using Method 1, then it is easy to see that a bound analogous to (14.23) holds for the right residual $A\hat{X} - I$. If X_L has a small left residual and X_U has a small right residual (or vice versa) then it does not seem possible to derive a bound of the form (14.23). However, we have

$$|X_L L - I| = |L^{-1}(L X_L - I)L| \leq |L^{-1}||L X_L - I||L|, \quad (14.24)$$

and since L is unit lower triangular with $|l_{ij}| \leq 1$, we have $|(L^{-1})_{ij}| \leq 2^{n-1}$, which places a bound on how much the left and right residuals of X_L can differ. Furthermore, since the matrices L from GEPP tend to be well conditioned ($\kappa_\infty(L) \ll n2^{n-1}$), and since our numerical experience is that large residuals tend to occur only for ill-conditioned matrices, we would expect the left and right residuals of X_L almost always to be of similar size. We conclude that even in the

“conflicting residuals” case, Method D will, in practice, usually satisfy (14.23) or its right residual counterpart, according to whether X_U has a small left or right residual respectively. Similar comments apply to Method B when U^{-1} is computed by a method yielding a small right residual.

These considerations are particularly pertinent when we consider Method D specialized to symmetric positive definite matrices and the Cholesky factorization $A = R^T R$. Now A^{-1} is obtained by computing $X_R = R^{-1}$ and then forming $A^{-1} = X_R X_R^T$; this is the method used in the LINPACK routine `xPODI` [341, 1979, Chap. 3]. If X_R has a small right residual then X_R^T has a small left residual, so in this application we naturally encounter conflicting residuals. Fortunately, the symmetry and definiteness of the problem help us to obtain a satisfactory residual bound. The analysis parallels the derivation of (14.23), so it suffices to show how to treat the term $X_R X_R^T R^T R$ (cf. (14.21)), where R now denotes the computed Cholesky factor. Assuming $RX_R = I + \Delta(R, X_R)$, and using (14.24) with L replaced by R , we have

$$\begin{aligned} X_R X_R^T R^T R &= X_R (I + \Delta(R, X_R)^T) R \\ &= I + F + X_R \Delta(R, X_R)^T R, \quad |F| \leq |R^{-1}| |\Delta(R, X_R)| |R|, \\ &= I + G, \end{aligned}$$

and

$$|G| \leq c_n u (|R^{-1}| |R| |R^{-1}| |R| + |R^{-1}| |R^{-T}| |R^T| |R|).$$

From the inequality $\| |B| \|_2 \leq \sqrt{n} \|B\|_2$ for $B \in \mathbb{R}^{n \times n}$ (see Lemma 6.6), together with $\|A\|_2 = \|R\|_2^2 + O(u)$, it follows that

$$\|G\|_2 \leq 2n^2 c_n u \|A\|_2 \|A^{-1}\|_2,$$

and thus overall we have a bound of the form

$$\|\hat{X}A - I\|_2 \leq d_n u \|A\|_2 \|\hat{X}\|_2.$$

Since $\hat{X}$ and A are symmetric the same bound holds for the right residual.

14.3.5. Summary

In terms of the error bounds, there is little to choose between Methods A, B, C, and D. Numerical results reported in [357, 1992] show good agreement with the bounds. Therefore the choice of method can be based on other criteria, such as performance and the use of working storage. Table 14.3 gives some performance figures for a Cray 2, covering both blocked (partitioned) and unblocked forms of Methods B, C, and D.

On a historical note, Tables 14.4 and 14.5 give timings for matrix inversion on some early computing devices; times for two more modern machines are given for comparison. The inversion methods used for the timings on the early computers in Table 14.4 are not known, but are probably methods from this section or the next.

Table 14.3. *Mflop rates for inverting a full matrix on a Cray 2.*

		$n = 64$	$n = 128$	$n = 256$	$n = 512$
Unblocked:	Method B	118	229	310	347
	Method C	125	235	314	351
	Method D	70	166	267	329
Blocked: (block size 64)	Method B	142	259	353	406
	Method C	144	264	363	415
	Method D	70	178	306	390

Table 14.4. *Times (minutes and seconds) for inverting an $n \times n$ matrix. Source for DEUCE, Pegasus, and Mark 1 timings: [200, 1981].*

n	DEUCE (English Electric) 1955	Pegasus (Ferranti) 1956	Manchester Mark 1 1951	HP 48G Calculator 1993	Sun SPARC- station ELC 1991
8	20s	26s	—	4s	.004s
16	58s	2m 37s	—	18s	.01s
24	3m 36s	7m 57s	8m	48s	.02s
32	7m 44s	17m 52s	16m	—	.04s

Table 14.5. *Additional timings for inverting an $n \times n$ matrix.*

Machine	Year	n	Time	Reference
Aiken Relay Calculator	1948	38	59½ hours	[858, 1948]
IBM 602 Calculating Punch	1949	10	8 hours	[1195, 1949]
SEAC (National Bureau of Standards)	1954	49	3 hours	[1141, 1954]
Datatron	1957	80 ^a	2½ hours	[847, 1957]
IBM 704	1957	115 ^b	19m 30s	[354, 1957]

^aBlock tridiagonal matrix, using an inversion method designed for such matrices.
^bSymmetric positive definite matrix.

14.4. Gauss–Jordan Elimination

Whereas Gaussian elimination (GE) reduces a matrix to triangular form by elementary operations, Gauss–Jordan elimination (GJE) reduces it all the way to diagonal form. GJE is usually presented as a method for matrix inversion, but it can also be regarded as a method for solving linear equations. We will take the method in its latter form, since it simplifies the error analysis. Error bounds for matrix inversion are obtained by taking unit vectors for the right-hand sides.

At the k th stage of GJE, all off-diagonal elements in the k th column are eliminated, instead of just those below the diagonal, as in GE. Since the elements in the lower triangle (including the pivots on the diagonal) are identical to those that occur in GE, Theorem 9.1 tells us that GJE succeeds if all the leading principal submatrices of A are nonsingular. With no form of pivoting GJE is unstable in general, for the same reasons that GE is unstable. Partial and complete pivoting are easily incorporated.

Algorithm 14.4 (Gauss–Jordan elimination). This algorithm solves the linear system $Ax = b$, where $A \in \mathbb{R}^{n \times n}$ is nonsingular, by GJE with partial pivoting.

```

for  $k = 1:n$ 
  Find  $r$  such that  $|a_{rk}| = \max_{i \geq k} |a_{ik}|$ .
   $A(k, k:n) \leftrightarrow A(r, k:n)$ ,  $b(k) \leftrightarrow b(r)$  % Swap rows  $k$  and  $r$ .
   $row\_ind = [1:k-1, k+1:n]$  % Row indices of elements to eliminate.
   $m = A(row\_ind, k)/A(k, k)$  % Multipliers.
   $A(row\_ind, k:n) = A(row\_ind, k:n) - m * A(k, k:n)$ 
  (*)  $b(row\_ind) = b(row\_ind) - m * b(k)$ 
end
 $x_i = b_i/a_{ii}$ ,  $i = 1:n$ 

```

Cost: n^3 flops.

Note that for matrix inversion we repeat statement (*) for $b = e_1, e_2, \dots, e_n$ in turn, and if we exploit the structure of these right-hand sides the total cost is $2n^3$ flops—the same as for inversion by LU factorization.

The numerical stability properties of GJE are rather subtle and error analysis is trickier than for GE. An observation that simplifies the analysis is that we can consider the algorithm as comprising two stages. The first stage is identical to GE and forms $M_{n-1}M_{n-2} \dots M_1 A = U$, where U is upper triangular. The second stage reduces U to diagonal form by elementary operations:

$$N_n N_{n-1} \dots N_2 U = D, \quad N_k = I - n_k e_k^T, \quad e_i^T n_k = 0, \quad i \geq k.$$

The solution x is formed as $x = D^{-1}N_n \dots N_2 y$, where $y = M_{n-1} \dots M_1 b$. The rounding errors in the first stage are precisely the same as those in GE, so we will ignore them for the moment (that is, we will consider L , U , and y to be exact) and consider the second stage. We will assume, for simplicity, that there are no row interchanges. As with GE, this is equivalent to assuming that all row interchanges are done at the start of the algorithm.

Define $U_{k+1} = N_k \dots N_2 U$ (so $U_2 = U$) and note that N_k and U_k have the forms

$$U_k = \begin{bmatrix} D_{k-1} & [v_k & W_k] \\ 0 & Z_k \end{bmatrix}, \quad D_{k-1} \in \mathbb{R}^{(k-1) \times (k-1)}, \text{ diagonal},$$

$$N_k = \begin{bmatrix} I_{k-1} & [n_k & 0] \\ 0 & I_{n-k+1} \end{bmatrix},$$

where $n_k = [-u_{1k}/u_{kk}, \dots, -u_{k-1,k}/u_{kk}]^T$. The computed matrices obviously satisfy

$$\widehat{U}_{k+1} = \widehat{N}_k \widehat{U}_k + \Delta_k, \quad (14.25a)$$

$$\Delta_k = \begin{bmatrix} 0_{k-1} & \Delta^{(k)} \\ 0 & 0_{n-k+1} \end{bmatrix}, \quad |\Delta_k| \leq \gamma_3 |\widehat{N}_k| |\widehat{U}_k| \quad (14.25b)$$

(to be precise, $\widehat{N}_k$ is defined as N_k but with $(\widehat{N}_k)_{ik} = -\widehat{u}_{ik}/\widehat{u}_{kk}$). Similarly, with $x_{k+1} = \widehat{N}_k \dots \widehat{N}_2 y$, we have

$$\widehat{x}_{k+1} = \widehat{N}_k \widehat{x} + f_k, \quad e_i^T f_k = 0, \quad i \geq k, \quad |f_k| \leq \gamma_3 |\widehat{N}_k| |\widehat{x}_k|. \quad (14.26)$$

Because of the structure of $\widehat{N}_k$, Δ_k , and f_k , we have the useful property that

$$\widehat{N}_i \Delta_j = \Delta_j, \quad i \geq j, \quad \widehat{N}_i f_j = f_j, \quad i \geq j.$$

Without loss of generality, we now assume that the final diagonal matrix D is the identity (i.e., the pivots are all 1); this simplifies the analysis a little and has a negligible effect on the final bounds. Thus (14.25) and (14.26) yield

$$I = \widehat{U}_{n+1} = \widehat{N}_n \dots \widehat{N}_2 U + \sum_{k=2}^n \Delta_k, \quad (14.27)$$

$$\widehat{x} = \widehat{x}_{n+1} = \widehat{N}_n \dots \widehat{N}_2 y + \sum_{k=2}^n f_k. \quad (14.28)$$

Now

$$\begin{aligned} |\Delta_k| &\leq \gamma_3 |\widehat{N}_k| |\widehat{U}_k| \leq \gamma_3 |\widehat{N}_k| |\widehat{N}_{k-1} \widehat{U}_{k-1} + \Delta_{k-1}| \\ &\leq \gamma_3 (1 + \gamma_3) |\widehat{N}_k| |\widehat{N}_{k-1}| |\widehat{U}_{k-1}| \\ &\leq \dots \leq \gamma_3 (1 + \gamma_3)^{k-2} |\widehat{N}_k| \dots |\widehat{N}_2| |U|, \end{aligned}$$

and, similarly,

$$|f_k| \leq \gamma_3 (1 + \gamma_3)^{k-2} |\widehat{N}_k| \dots |\widehat{N}_2| |y|.$$

But defining $\tilde{n}_k = [n_k^T \ 0]^T \in \mathbb{R}^n$, we have

$$\begin{aligned} |\widehat{N}_k| \dots |\widehat{N}_2| &= I + |\tilde{n}_k| e_k^T + \dots + |\tilde{n}_2| e_2^T \\ &\leq |\widehat{N}_n| \dots |\widehat{N}_2| = |\widehat{N}_n \dots \widehat{N}_2| =: |X|, \end{aligned}$$

where $X = U^{-1} + O(u)$ (by (14.27)). Hence

$$\begin{aligned}\sum_{k=2}^n |\Delta_k| &\leq (n-1)\gamma_3(1+\gamma_3)^{n-2}|X||U|, \\ \sum_{k=2}^n |f_k| &\leq (n-1)\gamma_3(1+\gamma_3)^{n-2}|X||y|.\end{aligned}$$

Combining (14.27) and (14.28) we have, for the solution of $Ux = y$,

$$\hat{x} = \left(I - \sum_{k=2}^n \Delta_k\right) U^{-1}y + \sum_{k=2}^n f_k = \left(I - \sum_{k=2}^n \Delta_k\right)x + \sum_{k=2}^n f_k,$$

which gives the componentwise forward error bound

$$|x - \hat{x}| \leq (n-1)\gamma_3(1+\gamma_3)^{n-2}|X|(|U||x| + |y|). \quad (14.29)$$

This is an excellent forward error bound: it says that the error in $\hat{x}$ is no larger than the error we would expect for an approximate solution with a tiny componentwise relative backward error. In other words, the forward error is bounded in the same way as for substitution. However, we have not, and indeed cannot, show that the method is backward stable. The best we can do is to derive from (14.27) and (14.28) the result

$$(U + \Delta U)\hat{x} = y + \Delta y, \quad (14.30a)$$

$$|\Delta U| \leq (n-1)\gamma_3(1+\gamma_3)^{n-2}|U||X||U|, \quad (14.30b)$$

$$|\Delta y| \leq (n-1)\gamma_3(1+\gamma_3)^{n-2}|U||X||y|, \quad (14.30c)$$

using $\hat{N}_2^{-1} \dots \hat{N}_n^{-1} = U + O(u)$. These bounds show that, with respect to the system $Ux = y$, $\hat{x}$ has a normwise backward error bounded approximately by $\kappa_\infty(U)(n-1)\gamma_3$. Hence the backward error can be guaranteed to be small only if U is well conditioned. This agrees with the comments of Peters and Wilkinson [938, 1975] that “the residuals corresponding to the Gauss–Jordan solution are often larger than those corresponding to back-substitution by a factor of order κ .”

A numerical example helps to illustrate the results. We take U to be the upper triangular matrix with 1s on the diagonal and -1 s everywhere above the diagonal ($U = U(1)$ from (8.3)). This matrix has condition number $\kappa_\infty(U) = n2^{n-1}$. Table 14.6 reports the normwise relative backward error $\eta_{U,b}(\hat{x}) = \|U\hat{x} - b\|_\infty / (\|U\|_\infty \|\hat{x}\|_\infty + \|b\|_\infty)$ (see (7.2)) for $b = Ux$, where $x = e/3$. Clearly, GJE is backward unstable for these matrices—the backward errors show a dependence on $\kappa_\infty(U)$. However, the relative distance between $\hat{x}$ and the computed solution from substitution (not shown in the table) is less than $\kappa_\infty(U)u$, which shows that the forward error is bounded by $\kappa_\infty(U)u$, confirming (14.29).

By bringing in the error analysis for the reduction to upper triangular form, we obtain an overall result for GJE.

Table 14.6. GJE for $Ux = b$.

n	$\eta_{U,b}(\hat{x})$	$\kappa_\infty(U)u$
16	2.0e-14	5.8e-11
32	6.4e-10	7.6e-6
64	1.7e-2	6.6e4

Theorem 14.5. Suppose GJE successfully computes an approximate solution $\hat{x}$ to $Ax = b$, where $A \in \mathbb{R}^{n \times n}$ is nonsingular. Then

$$|b - A\hat{x}| \leq 8nu|\hat{L}||\hat{U}||\hat{U}^{-1}||\hat{U}||\hat{x}| + O(u^2), \quad (14.31)$$

$$|x - \hat{x}| \leq 2nu(|A^{-1}||\hat{L}||\hat{U}| + 3|\hat{U}^{-1}||\hat{U}|)|\hat{x}| + O(u^2), \quad (14.32)$$

where $A \approx \hat{L}\hat{U}$ is the factorization computed by GE.

Proof. For the first stage (which is just GE), we have $A + \Delta A_1 = \hat{L}\hat{U}$, $|\Delta A_1| \leq \gamma_n|\hat{L}||\hat{U}|$, and $(\hat{L} + \Delta L)\hat{y} = b$, $|\Delta L| \leq \gamma_n|\hat{L}|$, by Theorems 9.3 and 8.5.

Using (14.30), we obtain

$$(\hat{L} + \Delta L)(\hat{U} + \Delta U)\hat{x} = (\hat{L} + \Delta L)(\hat{y} + \Delta y) = b + (\hat{L} + \Delta L)\Delta y,$$

or $A\hat{x} = b - r$, where

$$r = (\Delta A_1 + \hat{L}\Delta U + \Delta L\hat{U} + \Delta L\Delta U)\hat{x} - (\hat{L} + \Delta L)\Delta y. \quad (14.33)$$

The bound (14.31) follows. To obtain (14.32) we use (14.29) and incorporate the effects of the errors in computing y . Defining x_0 by $\hat{L}\hat{U}x_0 = b$, we have

$$x - \hat{x} = (x - x_0) + (x_0 - \hat{U}^{-1}\hat{y}) + (\hat{U}^{-1}\hat{y} - \hat{x}),$$

where the last term is bounded by (14.29) and the first two terms are easily shown to be bounded by $\gamma_n|A^{-1}||\hat{L}||\hat{U}||\hat{x}| + O(u^2)$. $\square$

Theorem 14.5 shows that the stability of GJE depends not only on the size of $|\hat{L}||\hat{U}|$ (as in GE), but also on the condition of U . The vector $|\hat{U}^{-1}||\hat{U}||\hat{x}|$ is an upper bound for $|\hat{x}|$, and if this bound is sharp then the residual bound is very similar to that for LU factorization. Note that for partial pivoting we have

$$\begin{aligned} \|\hat{U}^{-1}||\hat{U}|\|_\infty &\leq n\|A^{-1}\|_\infty \cdot n\rho_n\|A\|_\infty + O(u) \\ &= n^2\rho_n\kappa_\infty(A) + O(u), \end{aligned}$$

and, similarly, $\|A^{-1}||\hat{L}||\hat{U}|\|_\infty \leq n^2\rho_n\kappa_\infty(A)$, so (14.32) implies

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq 8n^3\rho_n\kappa_\infty(A)u + O(u^2),$$

which shows that GJE with partial pivoting is normwise forward stable.

The bounds in Theorem 14.5 have the pleasing property that they are invariant under row or column scaling of A , though of course if we are using partial pivoting then row scaling can change the pivots and alter the bound.

As mentioned earlier, to obtain bounds for matrix inversion we simply take b to be each of the unit vectors in turn. For example, the residual bound becomes

$$|A\hat{X} - I| \leq 8nu|\hat{L}||\hat{U}||\hat{U}^{-1}||\hat{U}||\hat{X}| + O(u^2).$$

The question arises of whether there is a significant class of matrices for which GJE is backward stable. The next result proves that GJE without pivoting is forward stable for symmetric positive definite matrices and provides a smaller residual bound than might be expected from Theorem 14.5. We make the natural assumption that symmetry is exploited in the elimination.

Corollary 14.6. *Suppose GJE successfully computes an approximate solution $\hat{x}$ to $Ax = b$, where $A \in \mathbb{R}^{n \times n}$ is symmetric positive definite. Then, provided the computed pivots are positive,*

$$\begin{aligned} \|b - A\hat{x}\|_2 &\leq 8n^3 u \kappa_2(A)^{1/2} \|A\|_2 \|x\|_2 + O(u^2), \\ \frac{\|x - \hat{x}\|_2}{\|x\|_2} &\leq 8n^{5/2} u \kappa_2(A) + O(u^2), \end{aligned}$$

where $A \approx \hat{L}\hat{U}$ is the factorization computed by symmetric GE.

Proof. By Theorem 9.3 we have $A + \Delta A = \hat{L}\hat{U}$, where ΔA is symmetric and satisfies $|\Delta A| \leq \gamma_n |\hat{L}||\hat{U}|$. Defining $D = \text{diag}(\hat{U})^{1/2}$ we have, by symmetry, $A + \Delta A = \hat{L}D \cdot D^{-1}\hat{U} \equiv R^T R$. Hence

$$\begin{aligned} \|\hat{U}^{-1}||\hat{U}|\|_2 &= \|R^{-1}D^{-1}||DR|\|_2 \\ &= \|R^{-1}||R|\|_2 \\ &\leq n\kappa_2(R) = n\kappa_2(A + \Delta A)^{1/2} \\ &= n\kappa_2(A)^{1/2} + O(u). \end{aligned}$$

Furthermore, it is straightforward to show that $\|\hat{L}||\hat{U}|\|_2 = \|R^T||R|\|_2 \leq n(1 - n\gamma_n)^{-1} \|A\|_2$. The required bounds follow (note that the first term on the right-hand side of (14.32) dominates). $\square$

Our final result shows that if A is row diagonally dominant then GJE without pivoting is both forward stable and row-wise backward stable (see Table 7.1).

Corollary 14.7. *If GJE successfully computes an approximate solution $\hat{x}$ to $Ax = b$, where $A \in \mathbb{R}^{n \times n}$ is row diagonally dominant, then*

$$\begin{aligned} |b - A\hat{x}| &\leq 32n^2 u |A|e e^T |\hat{x}| + O(u^2), \\ \frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} &\leq 4n^3 u (\kappa_\infty(A) + 3) + O(u^2). \end{aligned}$$

Proof. The bounds follow from Theorem 14.5 on noting that U is row diagonally dominant and using Lemma 8.8 to bound $\text{cond}(U)$ and (9.17) to bound $\|L||U|\|_\infty$. $\square$

14.5. Parallel Inversion Methods

Vavasis [1192, 1989] shows that GEPP is P-complete, a complexity result whose implication is that GEPP cannot be efficiently implemented on a highly parallel computer with a large number of processors. This result provides motivation for looking for non-GE-based methods for solving linear systems on massively parallel machines. In this section we briefly describe some matrix inversion methods whose computational complexity makes them contenders for linear system solution on parallel, and perhaps other, machines.

Csanky [285, 1976] gives a method for inverting an $n \times n$ matrix (and thereby solving a linear system) in $O(\log^2 n)$ time on $O(n^4)$ processors. This method has optimal computational complexity amongst known methods for massively parallel machines, but it involves the use of the characteristic polynomial and it has abysmal numerical stability properties (as can be seen empirically by applying the method to $3I$, for example!).

A more practical method is Newton's method for A^{-1} , which is known as the Schulz iteration [1027, 1933]:

$$X_{k+1} = X_k(2I - AX_k) = (2I - X_kA)X_k.$$

It is known that X_k converges to A^{-1} (or, more generally, to the pseudo-inverse if A is rectangular) if $X_0 = \alpha_0 A^T$ and $0 < \alpha_0 < 2/\|A\|_2^2$ [1056, 1974]. The Schulz iteration is attractive because it involves only matrix multiplication—an operation that can be implemented very efficiently on high-performance computers. The rate of convergence is quadratic, since if $E_k = I - AX_k$ or $E_k = I - X_kA$ then

$$E_{k+1} = E_k^2 = \cdots = E_0^{2^{k+1}}.$$

Like Csanky's method, the Schulz iteration has polylogarithmic complexity, but, unlike Csanky's method, it is numerically stable [1056, 1974]. Unfortunately, for scalar $a > 0$,

$$0 < x_k \ll a^{-1} \quad \Rightarrow \quad x_{k+1} = 2x_k - x_k a x_k \lesssim 2x_k,$$

and since $x_k \rightarrow a^{-1}$, convergence can initially be slow. Overall, about $2 \log_2 \kappa_2(A)$ iterations are required for convergence in floating point arithmetic. Pan and Schreiber [919, 1991] show how to accelerate the convergence by at least a factor 2 via scaling parameters and how to use the iteration to carry out rank and projection calculations. For an interesting application of the Schulz iteration to a sparse matrix possessing a sparse inverse, see [14, 1993], and for its adaptation to Toeplitz-like (low displacement rank) matrices see [917, 2001, Chap. 6] and [918, 1999].

For inverting triangular matrices a divide and conquer method with polylogarithmic complexity is readily derived. See Higham [605, 1995] for details and error analysis.

Another inversion method is one of Strassen, which makes use of his fast matrix multiplication method: see Problem 23.8 and §26.3.2.

14.6. The Determinant

*It may be too optimistic to hope that determinants will
fade out of the mathematical picture in a generation;
their notation alone is a thing of beauty
to those who can appreciate that sort of beauty.*

— E. T. BELL, *Review of "Contributions to the History of Determinants, 1900–1920", by Sir Thomas Muir (1931)*

Like the matrix inverse, the determinant is a quantity that rarely needs to be computed. The common fallacy that the determinant is a measure of ill conditioning is displayed by the observation that if $Q \in \mathbb{R}^{n \times n}$ is orthogonal then $\det(\alpha Q) = \alpha^n \det(Q) = \pm \alpha^n$, which can be made arbitrarily small or large despite the fact that αQ is perfectly conditioned. Of course, we could normalize the matrix before taking its determinant and define, for example,

$$\psi(A) := \frac{1}{\det(D^{-1}A)} = \frac{\det(D)}{\det(A)}, \quad D = \text{diag}(\|A(i, :)\|_2),$$

where $D^{-1}A$ has rows of unit 2-norm. This function ψ is called the Hadamard condition number by Birkhoff [112, 1975], because Hadamard's determinantal inequality (see Problem 14.11) implies that $\psi(A) \geq 1$, with equality if and only if A has mutually orthogonal rows. Unless A is already row equilibrated (see Problem 14.13), $\psi(A)$ does not relate to the conditioning of linear systems in any straightforward way.

As good a way as any to compute the determinant of a general matrix $A \in \mathbb{R}^{n \times n}$ is via GEPP. If $PA = LU$ then

$$\det(A) = \det(P)^{-1} \det(U) = (-1)^r u_{11} \dots u_{nn}, \quad (14.34)$$

where r is the number of row interchanges during the elimination. If we use (14.34) then, barring underflow and overflow, the computed determinant $\hat{d} = fl(\det(A))$ satisfies

$$\hat{d} = \det(\hat{U})(1 + \theta_n),$$

where $|\theta_n| \leq \gamma_n$, so we have a tiny relative perturbation of the exact determinant of a perturbed matrix $A + \Delta A$, where ΔA is bounded in Theorem 9.3. In other words, we have almost the right determinant for a slightly perturbed matrix (assuming the growth factor is small).

However, underflow and overflow in computing the determinant are quite likely, and the determinant itself may not be a representable number. One possibility is to compute $\log |\det(A)| = \sum_{i=1}^n \log |u_{ii}|$; as pointed out by Forsythe and Moler [431, 1967, p. 55], however, the computed sum may be inaccurate due to cancellation. Another approach, used in LINPACK, is to compute and represent $\det(A)$ in the form $y \times 10^e$, where $1 \leq |y| < 10$ and e is an integer exponent.

14.6.1. Hyman's Method

In §1.16 we analysed the evaluation of the determinant of a Hessenberg matrix H by GE. Another method for computing $\det(H)$ is Hyman's method [652, 1957], which is superficially similar to GE but algorithmically different. Hyman's method has an easy derivation in terms of LU factorization that leads to a very short error analysis. Let $H \in \mathbb{R}^{n \times n}$ be an unreduced upper Hessenberg matrix ($h_{i+1,i} \neq 0$ for all i) and write

$$H = \begin{bmatrix} h^T & \eta \\ T & y \end{bmatrix}, \quad H_1 = \begin{bmatrix} T & y \\ h^T & \eta \end{bmatrix}, \quad h, y \in \mathbb{R}^{n-1}, \eta \in \mathbb{R}.$$

The matrix H_1 is H with the first row cyclically permuted to the bottom, so $\det(H_1) = (-1)^{n-1} \det(H)$. Since T is a nonsingular upper triangular matrix, we have the LU factorization

$$H_1 = \begin{bmatrix} I & 0 \\ h^T T^{-1} & 1 \end{bmatrix} \begin{bmatrix} T & y \\ 0 & \eta - h^T T^{-1} y \end{bmatrix} \equiv LU, \quad (14.35)$$

from which we obtain $\det(H_1) = \det(T)(\eta - h^T T^{-1} y)$. Therefore

$$\det(H) = (-1)^{n-1} \det(T)(\eta - h^T T^{-1} y). \quad (14.36)$$

Hyman's method consists of evaluating (14.36) in the natural way: by solving the triangular system $Tx = y$ by back substitution, then forming $\eta - h^T x$ and its product with $\det(T) = h_{21}h_{32} \dots h_{n,n-1}$.

For the error analysis we assume that no underflows or overflows occur. From the backward error analysis for substitution (Theorem 8.5) and for an inner product (see (3.4)) we have immediately, on defining $\mu := \eta - h^T T^{-1} y$,

$$\hat{\mu} = (\eta - (h + \Delta h)^T (T + \Delta T)^{-1} y)(1 + \delta),$$

where

$$|\Delta h| \leq \gamma_{n-1} |h|, \quad |\Delta T| \leq \gamma_{n-1} |T|, \quad |\delta| \leq u.$$

The computed determinant therefore satisfies

$$\hat{d} := fl(\det(H)) = (1 + \theta_n) \det(T)(\eta - (h + \Delta h)^T (T + \Delta T)^{-1} y),$$

where $|\theta_n| \leq \gamma_n$. This is not a backward error result, because only one of the two T s in this expression is perturbed. However, we can write $\det(T) = \det(T + \Delta T)(1 + \theta_{(n-1)^2})$, so that

$$\hat{d} = \det(T + \Delta T)(\eta(1 + \theta_{n^2-n+1}) - (h + \Delta h)^T (T + \Delta T)^{-1} y(1 + \theta_{n^2-n+1})).$$

We conclude that the computed determinant is the exact determinant of a perturbed Hessenberg matrix in which each element has undergone a relative perturbation not exceeding $\gamma_{n^2-n+1} \approx n^2 u$, which is a very satisfactory result. In fact, the constant γ_{n^2-n+1} can be reduced to γ_{2n-1} by a slightly modified analysis; see Problem 14.14.

14.7. Notes and References

Classic references on matrix inversion are Wilkinson's paper *Error Analysis of Direct Methods of Matrix Inversion* [1229, 1961] and his book *Rounding Errors in Algebraic Processes* [1232, 1963]. In discussing Method 1 of §14.2.1, Wilkinson says "The residual of X as a left-hand inverse may be larger than the residual as a right-hand inverse by a factor as great as $\|L\| \|L^{-1}\| \dots$ We are asserting that the computed X is *almost invariably* of such a nature that $XL - I$ is equally small" [1232, 1963, p. 107]. Our experience concurs with the latter statement. Triangular matrix inversion provides a good example of the value of rounding error analysis: it helps us identify potential instabilities, even if they are rarely manifested in practice, and it shows us what kind of stability is guaranteed to hold.

In [1229, 1961] Wilkinson identifies various circumstances in which triangular matrix inverses are obtained to much higher accuracy than the bounds of this chapter suggest. The results of §8.2 provide some insight. For example, if T is a triangular M -matrix then, as noted after Corollary 8.11, its inverse is computed to high relative accuracy, no matter how ill conditioned L may be.

Sections 14.2 and 14.3 are based closely on Du Croz and Higham [357, 1992].

Method D in §14.3.4 is used by the Hewlett-Packard HP-15C calculator, for which the method's lack of need for extra storage is an important property [570, 1982].

GJE is an old method. It was discovered independently by the geodesist Wilhelm Jordan (1842–1899) (not the mathematician Camille Jordan (1838–1922)) and B.-I. Clasen [16, 1987].

An Algol routine for inverting positive definite matrices by GJE was published in the *Handbook for Automatic Computation* by Bauer and Reinsch [94, 1971]. As a means of solving a single linear system, GJE is 50% more expensive than GE when cost is measured in flops; the reason is that GJE takes $O(n^3)$ flops to solve the upper triangular system that GE solves in n^2 flops. However, GJE has attracted interest for vector computing because it maintains full vector lengths throughout the algorithm. Hoffmann [633, 1987] found that it was faster than GE on the CDC Cyber 205, a now-defunct machine with a relatively large vector startup overhead.

Turing [1166, 1948] gives a simplified analysis of the propagation of errors in GJE, obtaining a forward error bound proportional to $\kappa(A)$. Bauer [91, 1966] does a componentwise forward error analysis of GJE for matrix inversion and obtains a relative error bound proportional to $\kappa_\infty(A)$ for symmetric positive definite A . Bauer's paper is in German and his analysis is not easy to follow. A summary of Bauer's analysis (in English) is given by Meinguet [839, 1969].

The first thorough analysis of the stability of GJE was by Peters and Wilkinson [938, 1975]. Their paper is a paragon of rounding error analysis. Peters and Wilkinson observe the connection with GE, then devote their attention to the "second stage" of GJE, in which $Ux = y$ is solved. They show that each component of x is obtained by solving a lower triangular system, and they deduce that, for each i , $(U + \Delta U_i)x^{(i)} = y + \Delta y_i$, where $|\Delta U_i| \leq \gamma_n |U|$ and $|\Delta y_i| \leq \gamma_n |y|$, and where the i th component of $x^{(i)}$ is the i th component of $\hat{x}$. They then show that $\hat{x}$ has relative error bounded by $2n\kappa_\infty(U)u + O(u^2)$, but that $\hat{x}$ does not necessarily

have a small backward error. The more direct approach used in our analysis is similar to that of Dekker and Hoffmann [303, 1989], who give a normwise analysis of a variant of GJE that uses row pivoting (elimination across rows) and column interchanges.

Peters and Wilkinson [938, 1975] state that “it is well known that Gauss–Jordan is stable” for a diagonally dominant matrix. In the first edition of this book we posed proving this statement as a research problem. In answer to this question, Malyshev [811, 2000] shows that normwise backward stability is easily proved using Theorem 14.5, as shown in Corollary 14.7. Our observation that row-wise backward stability holds is new. Malyshev also shows that GJE without pivoting is backward unstable for matrices diagonally dominant by columns.

Goodnight [512, 1979] describes the use of GJE in statistics for solving least squares problems. Parallel implementation of GJE is discussed by Quintana, Quintana, Sun, and van de Geijn [966, 2001].

Demmel [316, 1993] points out the numerical instability of Csanky’s method. For discussion of the numerical properties of Csanky’s method and the Schulz iteration from a computational complexity viewpoint see Codenotti, Leoncini and Preparata [248, 2001].

For an application of the Hadamard condition number see Burdakov [184, 1997].

Error analysis of Hyman’s method is given by Wilkinson [1228, 1960], [1232, 1963, pp. 147–154], [1233, 1965, pp. 426–431]. Although it dates from the 1950s, Hyman’s method is not obsolete: it has found use in methods designed for high-performance computers; see Ward [1207, 1976], Li and Zeng [784, 1992], and Dubrulle and Golub [359, 1994].

Some fundamental problems in computational geometry can be reduced to testing the sign of the determinant of a matrix, so the question arises of how to be sure that the sign is determined correctly in floating point arithmetic. For work on this problem see Pan and Yu [920, 2001] and the references therein.

Intimately related to the matrix inverse is the adjugate matrix,

$$\text{adj}(A) = ((-1)^{i+j} \det(A_{ji})),$$

where A_{ij} denotes the submatrix of A obtained by deleting row i and column j . Thus the adjugate is the transpose of the matrix of cofactors of A , and for non-singular A , $A^{-1} = \text{adj}(A)/\det(A)$. Stewart [1081, 1998] shows that the adjugate can be well conditioned even when A is ill conditioned, and he shows how $\text{adj}(A)$ can be computed in a numerically stable way from a rank revealing decomposition of A .

14.7.1. LAPACK

LAPACK contains computational routines, but not high-level drivers, for matrix inversion. Routine `xGETRI` computes the inverse of a general square matrix by Method B using an LU factorization computed by `xGETRF`. The corresponding routine for a symmetric positive definite matrix is `xPOTRI`, which uses Method D, with a Cholesky factorization computed by `xPOTRF`. Inversion of a symmetric indefinite matrix is done by `xSYTRI`. Triangular matrix inversion is done by `xTRTRI`,

which uses Method 2C. None of the LAPACK routines computes the determinant, but it is easy for the user to evaluate it after computing an LU factorization.

Problems

14.1. Reflect on this cautionary tale told by Acton [4, 1970, p. 246].

It was 1949 in Southern California. Our computer was a very new CPC (model 1, number 1)—a 1-second-per-arithmetic-operation clunker that was holding the computational fort while an early electronic monster was being coaxed to life in an adjacent room. From a nearby aircraft company there arrived one day a 16×16 matrix of 10-digit numbers whose inverse was desired . . . We labored for two days and, after the usual number of glitches that accompany any strange procedure involving repeated handling of intermediate decks of data cards, we were possessed of an inverse matrix. During the checking operations . . . it was noted that, to eight significant figures, the inverse was the transpose of the original matrix! A hurried visit to the aircraft company to explore the source of the matrix revealed that each element had been laboriously hand computed from some rather simple combinations of sines and cosines of a common angle. It took about 10 minutes to prove that the matrix was, indeed, orthogonal!

14.2. Rework the analysis of the methods of §14.2.2 using the assumptions (13.4) and (13.5), thus catering for possible use of fast matrix multiplication techniques.

14.3. Show that for any nonsingular matrix A ,

$$\kappa(A) \geq \max \left\{ \frac{\|AX - I\|}{\|XA - I\|}, \frac{\|XA - I\|}{\|AX - I\|} \right\}.$$

This inequality shows that the left and right residuals of X as an approximation to A^{-1} can differ greatly only if A is ill conditioned.

14.4. (Mendelsohn [841, 1956]) Find parametrized 2×2 matrices A and X such that the ratio $\|AX - I\|/\|XA - I\|$ can be made arbitrarily large.

14.5. Let X and Y be approximate inverses of $A \in \mathbb{R}^{n \times n}$ that satisfy

$$|A\hat{X} - I| \leq u|A||\hat{X}| \quad \text{and} \quad |\hat{Y}A - I| \leq u|\hat{Y}||A|,$$

and let $\hat{x} = fl(\hat{X}b)$ and $\hat{y} = fl(\hat{Y}b)$. Show that

$$|A\hat{x} - b| \leq \gamma_{n+1}|A||\hat{X}||b| \quad \text{and} \quad |A\hat{y} - b| \leq \gamma_{n+1}|A||\hat{Y}||A||x|.$$

Derive forward error bounds for $\hat{x}$ and $\hat{y}$. Interpret all these bounds.

14.6. What is the inverse of the matrix on the front cover of the *LAPACK Users' Guide* [20, 1999]? (The first two editions gave the inverse on the back cover.) Answer the same question for the *LINPACK Users' Guide* [341, 1979].

14.7. Show that for any matrix having a row or column of 1s, the elements of the inverse sum to 1.

14.8. Let $X = A + iB \in \mathbb{C}^{n \times n}$ be nonsingular, where A and B are real. Show that X^{-1} can be expressed in terms of the inverse of the real matrix of order $2n$,

$$Y = \begin{bmatrix} A & -B \\ B & A \end{bmatrix}.$$

Show that if X is Hermitian positive definite then Y is symmetric positive definite. Compare the economics of real versus complex inversion.

14.9. For a given nonsingular $A \in \mathbb{R}^{n \times n}$ and $X \approx A^{-1}$, it is interesting to ask what is the smallest ϵ such that $X + \Delta X = (A + \Delta A)^{-1}$ with $\|\Delta X\| \leq \epsilon \|X\|$ and $\|\Delta A\| \leq \epsilon \|A\|$. We require $(A + \Delta A)(X + \Delta X) = I$, or

$$A\Delta X + \Delta AX + \Delta A\Delta X = I - AX.$$

It is reasonable to drop the second-order term, leaving a generalized Sylvester equation that can be solved using singular value decompositions of A and X (cf. §16.2). Investigate this approach computationally for a variety of A and methods of inversion.

14.10. For a nonsingular $A \in \mathbb{R}^{n \times n}$ and given integers i and j , under what conditions is $\det(A)$ independent of a_{ij} ? What does this imply about the suitability of $\det(A)$ for measuring conditioning?

14.11. Prove Hadamard's inequality for $A \in \mathbb{C}^{n \times n}$:

$$|\det(A)| \leq \prod_{k=1}^n \|a_k\|_2,$$

where $a_k = A(:, k)$. When is there equality? (Hint: use the QR factorization.)

14.12. (a) Show that if $A^T = QR$ is a QR factorization then the Hadamard condition number $\psi(A) = \prod_{i=1}^n \rho_i / |r_{ii}|$, where $\rho_i = \|R(:, i)\|_2$. (b) Evaluate $\psi(A)$ for $A = U(1)$ (see (8.3)) and for the Pei matrix $A = (\alpha - 1)I + ee^T$.

14.13. (Guggenheimer, Edelman, and Johnson [530, 1995]) (a) Prove that for a nonsingular $A \in \mathbb{R}^{n \times n}$,

$$\kappa_2(A) < \frac{2}{|\det(A)|} \left(\frac{\|A\|_F}{n^{1/2}} \right)^n. \quad (14.37)$$

(Hint: apply the arithmetic–geometric mean inequality to the n numbers $\sigma_1^2/2, \sigma_1^2/2, \sigma_2^2, \dots, \sigma_{n-1}^2$, where the σ_i are the singular values of A .) (b) Deduce that if A has rows of unit 2-norm then

$$\kappa_2(A) < \frac{2}{|\det(A)|} = 2\psi(A),$$

where ψ is the Hadamard condition number. For more on this type of bound see [842, 1997], and for a derivation of (14.37) from a different perspective see [336, 1993].

14.14. Show that Hyman's method for computing $\det(H)$, where $H \in \mathbb{R}^{n \times n}$ is an unreduced upper Hessenberg matrix, computes the exact determinant of $H + \Delta H$ where $|\Delta H| \leq \gamma_{2n-1}|H|$, barring underflow and overflow. What is the effect on the error bound of a diagonal similarity transformation $H \rightarrow D^{-1}HD$, where $D = \text{diag}(d_i)$, $d_i \neq 0$?

14.15. (Godunov, Antonov, Kiriljuk, and Kostin [493, 1993]) Show that if $A \in \mathbb{R}^{n \times n}$ and $\kappa_2(A)\|\Delta A\|_2/\|A\|_2 < 1$ then

$$\frac{|\det(A + \Delta A) - \det(A)|}{\det(A)} \leq \frac{n\kappa_2(A) \frac{\|\Delta A\|_2}{\|A\|_2}}{1 - n\kappa_2(A) \frac{\|\Delta A\|_2}{\|A\|_2}}.$$